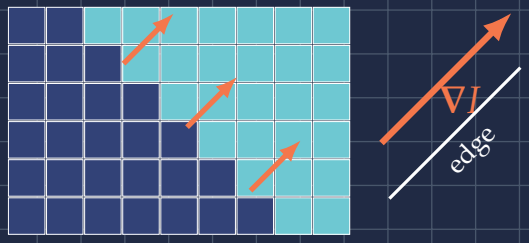
FROM PIXELS TO EDGE DIRECTION

The Scharr Gradient

A visual guide to discrete image derivatives

Kernels, convolution, magnitude, orientation,
and rotational accuracy

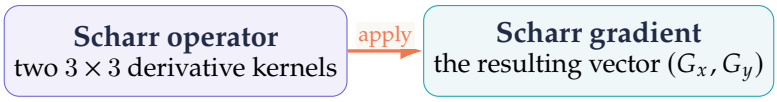


A picture-first explanation with exact calculations

$I \longmapsto (G_x, G_y) \longmapsto (G, \theta)$

What is the Scharr gradient?

The phrase combines two related objects:

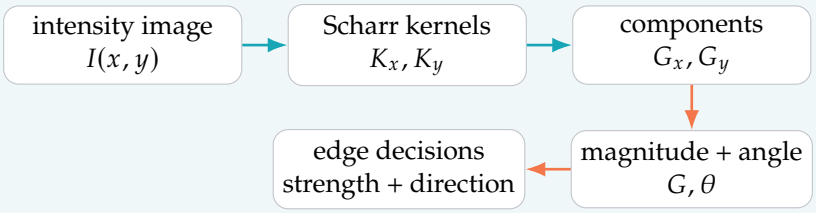


At each pixel, the operator estimates how quickly brightness changes horizontally and vertically. Those two estimates form a vector:

$$\nabla I \approx (G_x, G_y).$$

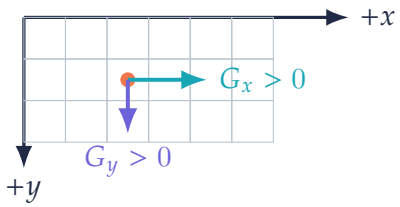
The vector points toward increasing brightness; its length measures edge strength.

The entire story in one pipeline



Coordinate convention used here

Images are arrays. We take x to increase to the right and y to increase *downward*, matching pixel row coordinates:



With this convention, $\operatorname{atan2}(G_y, G_x)$ increases clockwise on a displayed image. To use ordinary Cartesian angles, negate the vertical component: $G_y^{\text{cart}} = -G_y$.

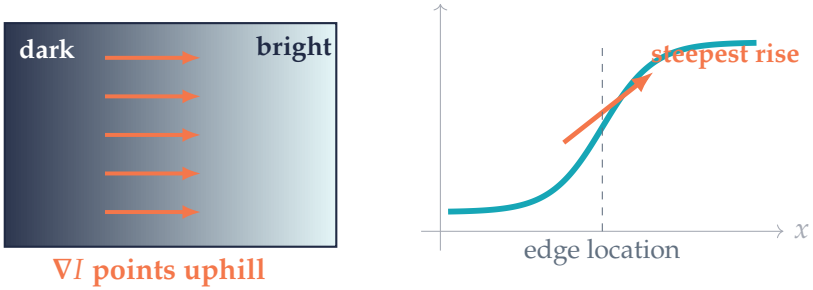
Name to use

Say *Scharr operator* for the filter pair, *Scharr derivatives* for G_x and G_y, and *Scharr gradient* for the vector field they form.

Before pixels: what a gradient means

Think of image intensity $I(x, y)$ as height above the image plane. A bright region is high; a dark region is low. The continuous gradient is

$$\nabla I = \left(\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right).$$



Three geometric facts

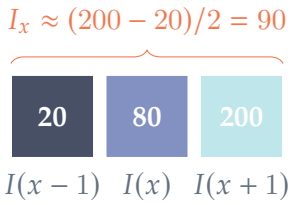
1. ∇I points in the direction of fastest brightness increase.
2. $\|\nabla I\|$ is large where brightness changes sharply.
3. ∇I is perpendicular to a smooth constant-intensity contour.

From derivatives to differences

Pixels do not give an infinitesimal neighborhood, so a discrete derivative uses nearby samples. In one dimension,

$$I_x(x) \approx \frac{I(x+1) - I(x-1)}{2}.$$

The stencil $[-1, 0, 1]/2$ compares the right side with the left side. Scharr performs this comparison across three rows at once, while weighting the rows to improve directional balance.



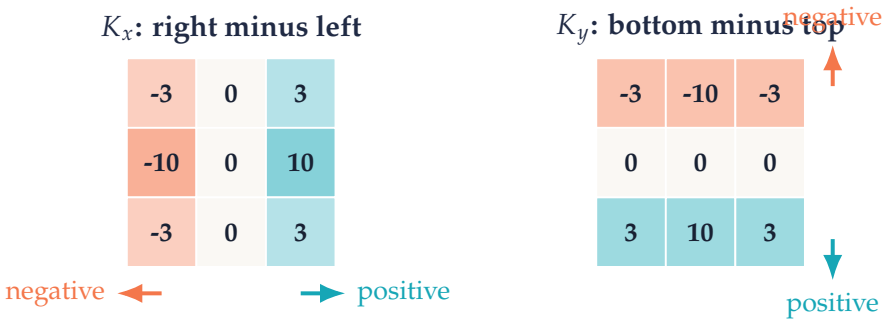
The two Scharr kernels

For a 3×3 neighborhood, the normalized kernels are

Scharr derivative pair

$$K_x = \frac{1}{32}\begin{bmatrix} -3 & 0 & 3 \\ -10 & 0 & 10 \\ -3 & 0 & 3 \end{bmatrix}, \qquad K_y = \frac{1}{32}\begin{bmatrix} -3 & -10 & -3 \\ 0 & 0 & 0 \\ 3 & 10 & 3 \end{bmatrix}.$$

Rows are shown from top to bottom. With the image-coordinate convention, K_x is positive for brightness increasing to the right, and K_y is positive for brightness increasing downward.



What one output pixel means

Using the correlation convention common in imaging libraries,

$$G_x(i,j) = \sum_{v=-1}^{1}\sum_{u=-1}^{1} K_x(v,u)I(i+v, j+u),$$

and similarly for G_y. Slide the kernel across the image, multiply nine aligned entries, and add. Strict mathematical convolution flips the kernel and therefore reverses the sign; edge strength is unchanged, but direction shifts by $180°$.

Normalization

Some software returns the raw integer response using weights 3 and 10. Dividing by 32 makes the response exact for a linear intensity ramp. Any common positive scale changes magnitudes but not gradient directions.

Why the kernel has this shape

The Scharr kernels are separable outer products:

$$K_x = \frac{1}{32}\begin{bmatrix}3\\10\\3\end{bmatrix}\begin{bmatrix}-1 & 0 & 1\end{bmatrix}, \qquad K_y = \frac{1}{32}\begin{bmatrix}-1\\0\\1\end{bmatrix}\begin{bmatrix}3 & 10 & 3\end{bmatrix}.$$

For K_x, each row performs a left-right difference. The middle row receives weight 10 and the outer rows weight 3. This fuses a small amount of vertical smoothing with a horizontal derivative.

Why 3 : 10 : 3 rather than 1 : 2 : 1?

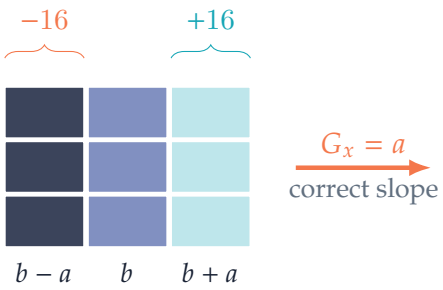
The Sobel operator uses the perpendicular weights 1 : 2 : 1. Scharr's 3 : 10 : 3 weights were chosen to reduce directional error for a 3×3 derivative. The result responds more evenly when the same edge rotates through the pixel grid.

A linear ramp calibration

Let $I(x, y) = ax + b$ be constant in y. The left column of a neighborhood has value $b - a$, the right column $b + a$, and their total weights are -16 and 16:

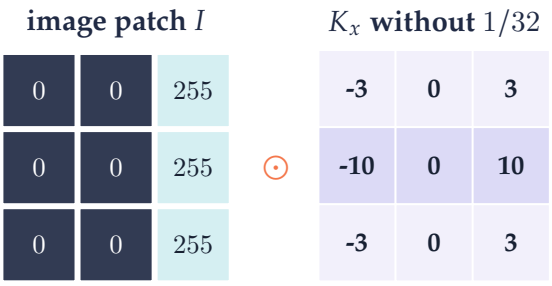
$$G_x = \frac{-16(b-a) + 16(b+a)}{32} = a, \qquad G_y = 0.$$

So the normalized kernel returns the exact horizontal slope.



Worked example: a vertical step edge

Consider a 3×3 patch that jumps from black to white at the right column:

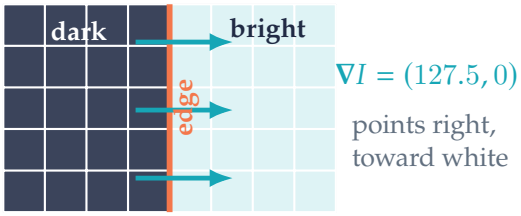


Only the right kernel column meets nonzero pixels:

$$G_x = \frac{3(255) + 10(255) + 3(255)}{32} = \frac{4080}{32} = 127.5.$$

For K_y, equal and opposite top and bottom contributions cancel:

$$G_y = 0.$$



Convert components to polar form

$$G = \sqrt{G_x^2 + G_y^2} = 127.5, \qquad \theta = \operatorname{atan2}(G_y, G_x) = 0^\circ.$$

The gradient is perpendicular to the vertical edge. Reversing black and white would reverse the vector and add 180° to its direction, while preserving its magnitude.

Worked example: a diagonal edge

Now let brightness rise toward the lower right:

$$I = \begin{bmatrix} 0 & 0 & 255 \\ 0 & 255 & 255 \\ 255 & 255 & 255 \end{bmatrix}.$$

The horizontal response is

$$G_x = \frac{3(255) + 10(255) - 3(255) + 3(255)}{32} = \frac{3315}{32} \approx 103.6.$$

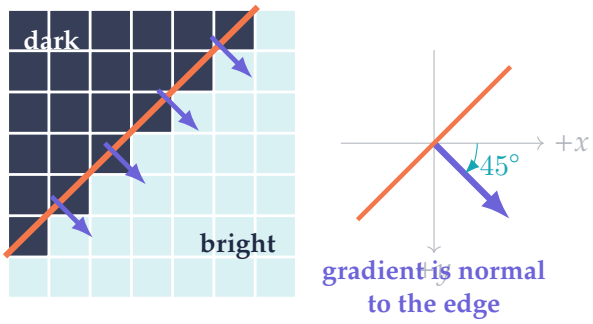
The vertical response is identical by symmetry:

$$G_y \approx 103.6.$$

Therefore

$$G \approx 146.5, \qquad \theta = 45^\circ$$

in image coordinates: down and to the right.



Edge direction versus gradient direction

The gradient angle describes the normal to the edge. The edge tangent is rotated by 90°:

$$\theta_{\text{edge}} = \theta_{\text{gradient}} + 90^\circ \pmod{180^\circ}.$$

Using modulo 180° treats an unoriented line as the same edge in either direction.

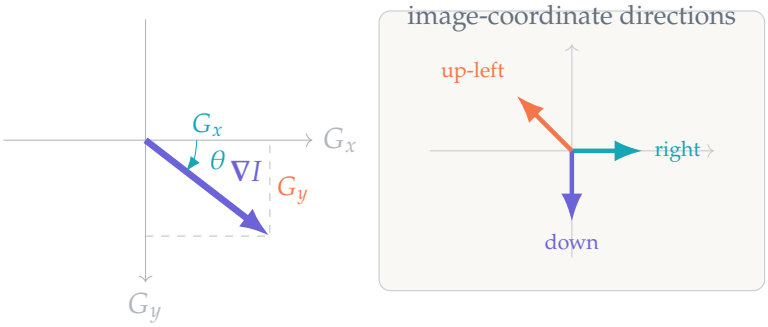
Magnitude and direction maps

Applying Scharr everywhere produces two signed images, G_x and G_y. They are best understood as components of one vector field.

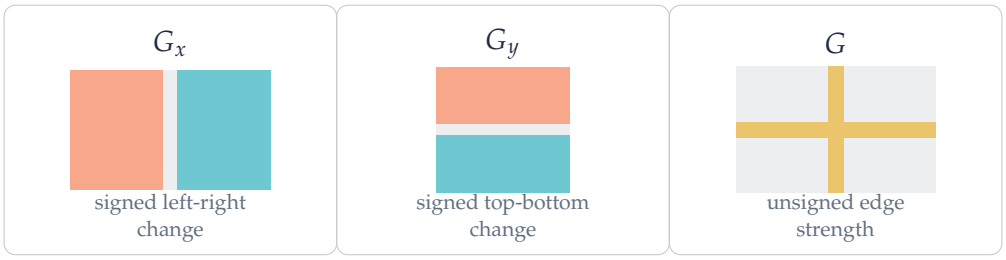
Polar representation

$$\boxed{G = \sqrt{G_x^2 + G_y^2}}, \qquad \boxed{\theta = \operatorname{atan2}(G_y, G_x)}.$$

An inexpensive approximation sometimes used in real-time systems is $G_1 = |G_x| + |G_y|$.



What the component images look like



Always store G_x and G_y in a signed floating-point or sufficiently wide signed integer type. An unsigned 8-bit image clips every negative response and destroys half the directional information.

Why Scharr improves on Sobel at 3×3

The normalized horizontal kernels are

$$K_x^{\text{Sobel}} = \frac{1}{8}\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \qquad K_x^{\text{Scharr}} = \frac{1}{32}\begin{bmatrix} -3 & 0 & 3 \\ -10 & 0 & 10 \\ -3 & 0 & 3 \end{bmatrix}.$$

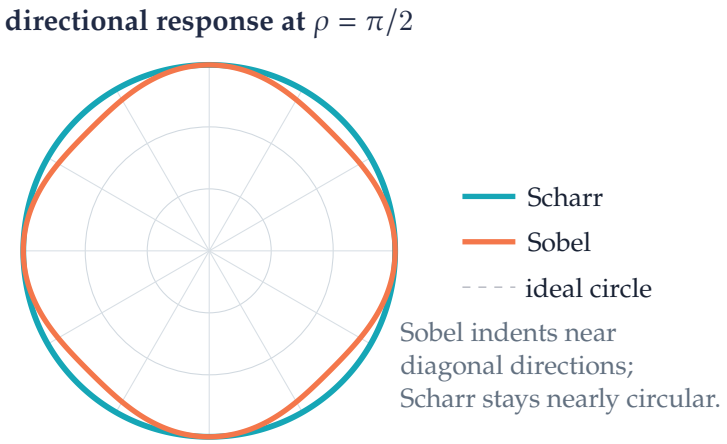
Both are exact on a linear ramp. The difference appears when an edge is diagonal or when the image contains shorter spatial wavelengths.

A frequency-response picture

For a sinusoidal image with angular-frequency vector (ω_x, ω_y), the Scharr responses are

$$H_x = \frac{i \sin \omega_x \,(10 + 6\cos \omega_y)}{16}, \qquad H_y = \frac{i \sin \omega_y \,(10 + 6\cos \omega_x)}{16}.$$

If the operator were perfectly rotation-neutral, total response at a fixed frequency would trace a circle as the direction rotates.



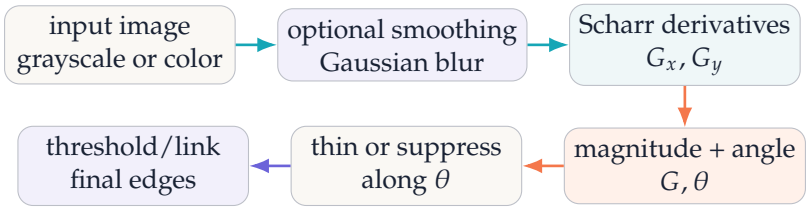
Interpretation

At this representative frequency, Scharr's magnitude varies much less as the same pattern rotates. It is therefore usually the better 3×3 choice when edge orientation matters. This is an accuracy improvement, not extra denoising.

What Scharr does not solve

Scharr still has only a 3×3 view. Strong noise, texture, aliasing, or a need for scale-selective edges may call for Gaussian smoothing, larger derivative kernels, or derivatives of Gaussians.

A practical edge pipeline



Implementation choices that matter

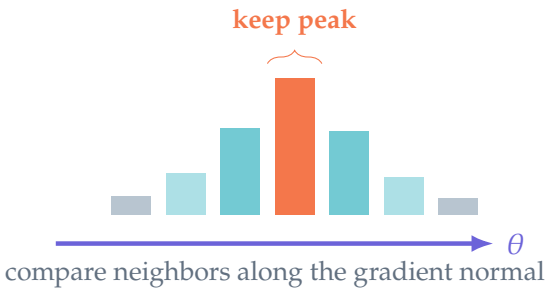
choice	practical consequence
output type	Use signed float or signed 16/32-bit storage; never clip derivatives to unsigned 8-bit.
kernel scale	Raw Scharr values are 32 times the normalized derivative used here. Keep one scale consistently.
border policy	Reflect padding usually avoids artificial high-contrast frames better than zero padding.
pre-smoothing	Derivatives amplify noise. A small Gaussian blur trades localization for stability.
magnitude norm	$\sqrt{G_x^2 + G_y^2}$ is rotation-neutral; $\lvert G_x\rvert + \lvert G_y\rvert$ is faster but directionally biased.
color handling	Grayscale can miss purely chromatic edges. For color-critical work, combine channel gradients or use a color-gradient method.

Library-shaped pseudocode

```
gx = scharr_x(image, signed_float, scale = 1/32)
gy = scharr_y(image, signed_float, scale = 1/32)
magnitude = sqrt(gx*gx + gy*gy)
angle = atan2(gy, gx)          # clockwise in image coordinates
```

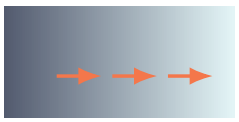
Non-maximum suppression

Magnitude alone makes thick bands around edges. To obtain a thin edge, look in the gradient direction and keep a pixel only if its magnitude is locally maximal along that line.



Reading results without fooling yourself

smooth shading



nonzero gradient, not necessarily an object edge

noise + texture

many strong local responses; smooth or reason at scale

bright line

two edges with opposite gradient directions

Common interpretation errors

- Calling θ the edge direction; it is the edge *normal*.
- Discarding negative derivatives before computing magnitude.
- Comparing raw Scharr magnitudes with normalized Sobel values.
- Treating every strong gradient as a semantic boundary.
- Ignoring the image-coordinate sign of y.
- Expecting a 3×3 kernel to suppress substantial noise.

Scale is part of the question

An edge is not an absolute object. At one scale, fabric texture creates hundreds of edges; after smoothing, the garment silhouette may dominate. Scharr estimates the derivative at the pixel scale presented to it.

Where Scharr fits

Scharr components are useful for edge maps, feature descriptors, focus measures, shape analysis, and gradient-based registration. They also serve as the derivative stage inside larger vision pipelines. These are local measurements, not an object detector or a complete segmentation method.

The reliable reading

A Scharr result says: "near this pixel, intensity rises this strongly in this direction." Everything beyond that - whether the change is noise, shading, texture, or an object boundary - requires context.

Compact reference sheet

Kernels

$$K_x = \frac{1}{32}\begin{bmatrix} -3 & 0 & 3 \\ -10 & 0 & 10 \\ -3 & 0 & 3 \end{bmatrix}$$

$$K_y = \frac{1}{32}\begin{bmatrix} -3 & -10 & -3 \\ 0 & 0 & 0 \\ 3 & 10 & 3 \end{bmatrix}$$

Coordinates: x right, y down.
Positive G_x: brighter to the right.
Positive G_y: brighter below.

Gradient quantities

$$\nabla I \approx (G_x, G_y)$$

$$G = \sqrt{G_x^2 + G_y^2}$$

$$\theta = \operatorname{atan2}(G_y, G_x)$$

$$\theta_{\text{edge}} = \theta + 90^\circ \pmod{180^\circ}$$

Cartesian display: use $(G_x, -G_y)$.
Raw kernels: multiply normalized values by 32.

Separable view

$$K_x = \frac{1}{32}\begin{bmatrix} 3 \\ 10 \\ 3 \end{bmatrix}\begin{bmatrix} -1 & 0 & 1 \end{bmatrix}.$$

Smooth perpendicular to the derivative, then compare opposite sides.

Three short exercises

1. A patch is constant along columns but changes from 20 on the top row to 220 on the bottom row. What signs do G_x and G_y have?
2. If $(G_x, G_y) = (-30, 40)$, compute the magnitude and describe the direction in image coordinates.
3. A vertical edge changes from white on the left to black on the right. What happens to the gradient direction compared with the worked example?

Answers

1. $G_x = 0$ and $G_y > 0$: brightness increases downward.
2. $G = 50$. The vector points down-left; the image-coordinate angle is approximately 126.9° clockwise from the positive x axis.
3. The magnitude is unchanged, but G_x becomes negative and the gradient points left - toward the brighter side.

Further reading

H. Scharr, *Optimal Operators in Digital Image Processing*; R. Szeliski, *Computer Vision: Algorithms and Applications*; and R. C. Gonzalez and R. E. Woods, *Digital Image Processing*.